

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_684a0618-ccb\agent-a0b8a9a05abf79656.jsonl`  
- SHA-256: `f7fc843772d992993cf43a72c9b7e67b9063becb1e44f4f7cf85061762af03f8`  
- Source modified: `2026-06-19T19:18:55+00:00`  
- Imported at: `2026-07-08T16:00:03+00:00`  
- project: `wf_684a0618-ccb`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-19T19:17:29.034Z)

THE IDEA TO ASSESS: a desktop app OR web UI where the user points KhelSutra at an ONLINE storage location they already own/pay for ? a Google Cloud Storage bucket, Google Drive, OneDrive/Microsoft 365, Dropbox, S3, etc. KhelSutra then WATCHES that folder, pulls/streams new recordings as they arrive, and AUTO-prepares highlights (same processing as the local/desktop path). "Bring your own storage + autosync": footage lands in their cloud ? reels appear automatically, zero manual upload.

You are assessing how this idea fits ONE area of the KhelSutra (badminton-highlight-indexer) docs. Read: docs/PLATFORM_ARCHITECTURE.md (focus S3 StorageBackend + ComputeBackend registries). CLAUDE.md says StorageBackend + ComputeBackend registries are the planned extension. Does the doc anticipate GCS/Drive/OneDrive/S3 connectors as StorageBackends, and a sync/ingest component? How would "watch a bucket ? enqueue job" fit the async job model (DEFERRED_HARDENING_PLAN #16, single-tenant)?

Be evidence-based and honest ? quote real anchors, don't invent. If the doc does NOT anticipate the idea, say so plainly and note what it WOULD take. Return the structured object.

2. assistant (2026-06-19T19:17:32.464Z)

I'll assess how the BYO-storage + autosync idea fits the KhelSutra platform architecture. Let me start by reading the relevant docs, pulling first per the repo-freshness convention.

3. user (2026-06-19T19:17:36.286Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

4. user (2026-06-19T19:17:36.788Z)

```
1      # Platform Architecture ? Multi-User "Assistant Coach"  
2  
3      **Status:** ? Strategy / research ? for review. No code yet.  
4      **Date:** 2026-06-07 . **Owner:** avidullu  
5  
6      > ? **Superseded where it disagrees with  
7      [OWNED_MODEL_IMPLEMENTATION_PLAN.md](OWNED_MODEL_IMPLEMENTATION_PLAN.md)  
8      > (2026-06-09, authoritative).** Specifically: (1) licensing is **MIT/Apache-2.0/BSD**  
9      (not "Apache-2.0 only");  
10     > (2) base model is **Qwen3-VL (Apache)** primary / InternVL3 (MIT) fallback ? **Gemma-4  
11     is AMBER/bench-only**, not a  
12     > cleared base; (3) the owned VLM is **not** the per-turn "conversational brain" ?  
13     extraction (TAL head ? later VLM)  
14     > feeds a per-video **event store + text-to-SQL** query layer ("extract once, query  
15     many"); the VLM-answers-each-turn
```

```

11     > design is rejected. The $5A model/base/QA-harness detail below predates that and is
kept for context only.
12
13     **Purpose.** Map the path from today's **single-user, local** tool to a **multi-tenant
14     "assistant coach" platform**: users upload videos, get highlights **stored and
15     browsable by history**, and choose **where their data lives** (Google Drive / cloud
16     bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
17     principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
18     execution/compute, the detection+highlight pipeline, and data-serving ? behind small
19     **pluggable interfaces**, reusing the registry pattern the codebase already uses for
20     segmenters/providers/validators/sports/annotations.
21
22     **Product framing (the north star *and* the v1 line).** This is an **assistant coach**,
23     not just a clipper ? but the strategy is **phased and unambiguous about scope**:
24     - **v1 (concrete, near-term):** **rally detection ? highlight-reel compilation ? per-
user
25     history ? a coach/user correction loop that feeds the golden set** ("assistant-coach
26     *lite"). Essentially today's pipeline made multi-user ? this is the real v1
deliverable.
27     - **Mid-term:** **rich rally/shot semantic metadata** (shot type, rally outcome) and the
28     **owned video-QA model** ($5A; $6 P4?P5).
29     - **North-star vision (strictly future ? *not* v1):** **gait / biomechanics, per-athlete
30     identifying templates, and deep coaching feedback under a supervising coach** ($6 P6).
31     Inspiring, but explicitly deferred ? and the part that raises the **biometric**
privacy
32     bar ($7).
33
34     The buyer is the **academy/coach** (beachhead in
35     [PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md)); **power users self-serve**. Two
36     consequences ripple through the design: (1) the data model keeps an **optional coach ?
37     athlete overlay over a generic core** ($4.1) ? not baked in; (2) gait/biometrics is
38     **special-category data**, gated and deferred ($7).
39
40     > **Competitive note (validated by research):** none of Veo / Hudl / SwingVision /
41     > Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
42     > (SwingVision uniquely runs *edge* compute on the user's iPhone). So **"your video,
43     > your storage, your compute" is a genuine differentiator for privacy-sensitive
44     > academies ? and SwingVision proves user-side compute is commercially viable. All four
45     > share the same **teams ? athletes ? clips** data model, which we adopt ($4).
46
47     > Scope: this is a **direction-setting doc**. It does not start implementation; the
48     > first infra slice (an async job model) is the on-hold #16 in
49     > [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md). Commercial constraints
come
50     > from [COMMERCIALIZATION.md](COMMERCIALIZATION.md) (C4 paid-LLM no-train; C12 SaaS
51     > foundations) and `archives/MONETIZATION_AUDIT.md`.
52
53     ## 1. Current state & gaps (what we're evolving from)
54
55     | Concern | Today | Gap for multi-user |
56     |---|---|---|
57     | Identity / tenancy | none | no users, orgs, auth, or isolation |
58     | Storage | local `filepath` string; output to `output_dir` | no abstraction; can't BYO
Drive/bucket; raw video must be on the box |
59     | Data model | `videos` keyed by file **MD5**, `play_segments`, `events`,
`candidate_segments` (SQLite) | no owner/athlete link; **highlights aren't a first-class
persisted entity** (compile writes a file and returns a path); no per-user history |
60     | Execution | **async job model shipped** ? `/api/process` returns **202 + job_id**, a
single-worker `ThreadPoolExecutor` drains the `jobs` table, `GET /api/jobs/{id}` polls status |
single-tenant + local-only; can't dispatch to a user's VM/GPU yet |
61     | Pipeline | **already pluggable** (segmenter/provider/validator/sport/annotation
registries) | reuse as-is ? this layer is the model for the rest |
62     | API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs
auth, per-tenant scoping, async job + history endpoints |
63

```

```

64      Grounding: `backend/infrastructure/database.py` (schema + `PRAGMA user_version`
65      migration ladder, **v5** ? `jobs` table + per-user `user_models` already landed),
66      `backend/main.py` (endpoints + the async job worker ? `/api/process` ? 202 + job_id,
67      `GET /api/jobs/{id}`), the registry pattern in
68      `backend/pipeline/segmenters/base.py::SegmenterRegistry` and
69      `backend/providers/base.py`.
70
71  ## 2. Target layered architecture
72
73  Decouple into layers with a clean control-plane vs data-plane split ? the
74  orchestrator coordinates metadata and jobs; the heavy bytes and GPU work can live
75  elsewhere (even in the user's own cloud).
76
77  ...
78
79      CONTROL      ? Presentation / API (auth, per-tenant) ?
80      PLANE        ? Identity & Tenancy (org?coach?athlete?user) ?
81                  ? Orchestration / Jobs (submit?status?result) ?
82                  ? dispatch      ? metadata
83
84      DATA        ? Execution / Compute ? ? Data / Serving ?
85      PLANE        ? (local | BYO-VM | ? ? (per-tenant DB: ?
86                  ? hosted GPU)      ? ? videos, segments, ?
87                  ? runs the Pipeline: ? ? highlights, jobs, ?
88                  ? detect ? highlight ? ? history) ?
89                  ? read input / write artifacts
90                  ? Storage (local upload | Google Drive | S3 | ?
91                  ? GCS | Azure) ? raw video + highlight outputs ?
92                  ?
93  ...
94
95  The two new pluggable seams are Storage and Compute/Execution; everything
96  else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A
97  model/training layer ? the owned generative video-QA model ($5A) ? sits atop these,
98  fed by Storage (training corpus), Data (labels), and Compute (training + inference), and
99  is delivered through the existing AI-provider registry with a paid fallback.
100
101  ## 3. Pluggable abstractions (extend the existing registry pattern)
102
103  The codebase already proves the model: an `ABC` + a `Registry` singleton with
104  `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with
105  the same shape so they're plug-and-play and testable in isolation.
106
107  ### 3.1 `StorageBackend` registry
108  Replace the bare `video_path: str` that flows through the pipeline with a
109  StorageRef (a backend id + a key/URI), resolved by a backend.
110
111  ```python
112  class StorageBackend(ABC):
113      name: str
114      def open_read(self, ref: StorageRef) -> BinaryIO: ... # stream large video
115      def download_to_local(self, ref: StorageRef, dst: str): ... # for ffmpeg/cv2/WSL
116      def upload(self, local_path: str, ref: StorageRef): ... # persist
117      highlight/artifacts
118      def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
119      def stat(self, ref: StorageRef) -> StorageStat: ...
120      def list(self, prefix: StorageRef) -> list[StorageRef]: ...
121
122  ...
123
124  - Recommended implementation: build the backend over fsspec (one matrix ?
125  local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`,
126  Dropbox via `dropboxdrivefs`), reaching for smart_open as the robust large-MP4

```

```

127     byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
128     add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)
129     - Backends: `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. BYO-storage:
130     the tenant registers their own bucket/Drive; raw video never leaves their account.
131     - Auth ? never persist users' long-lived keys. For buckets, have the user create an
132     IAM role you AssumeRole into, scoped to one bucket/prefix, and mint presigned
133     URLs so raw video moves browser ? the user's bucket directly, never through our
134     servers. For Drive, OAuth with the narrow drive.file scope; for academies, a
135     service account with domain-wide delegation. Store any per-tenant creds encrypted.
136     - Why it matters for the pipeline: ffmpeg/OpenCV/WSL need a local file, while
the
137     LLM provider can take a signed URL. The backend exposes both download_to_local
138     (for compute) and signed_url (for the provider/browser), so the pipeline stops
caring
139     where the bytes live.
140     - Migration path: introduce StorageRef alongside the current local path; a local
141     backend wraps today's behavior so nothing breaks; adapters land incrementally.
142
143     ### 3.2 `ComputeBackend` / `ExecutionBackend` registry
144     Where the pipeline actually runs for a given job.
145
146     - local ? in-process / local subprocess (today's behavior; dev + self-host).
147     - hosted ? our GPU VM pool (the SaaS default).
148     - byo_worker ? the tenant runs a pull-based worker (a small agent) on their
own
149     VM/GPU. Critical security property: the worker pulls jobs scoped to its tenant
150     from the control plane, pulls input from the tenant's own storage, runs the
151     pipeline, and writes artifacts + results back ? so the orchestrator never holds the
152     user's cloud credentials and needs no inbound access to their network. This
153     mirrors the self-hosted CI runner pattern (GitHub Actions runners use a 50s
154     outbound-only HTTPS long-poll). This pull-based runner is the cheapest, most
155     credential-safe v1 for BYO-compute.
156     - For users who want the platform to provision GPUs in their own cloud account on
157     demand (an academy with an AWS account but no standing GPU box), add SkyPilot
158     (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
159     Modal is the managed fallback for users wanting zero infra (accepting that raw
160     video then flows through Modal).
161     - Operational / support burden (a real adoption risk). For the primary beachhead
?
162     Indian badminton academies/coaches ? asking them to run and maintain a worker on
163     their own VM/GPU is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
164     "it stopped after our IT change" ticket). So: default academy users to `hosted`,
and
165     treat byo_worker/SkyPilot as a power-user / enterprise / privacy-sensitive tier,
166     not the academy default. If/when BYO ships, plan for auto-updating, self-healing
167     agents + health heartbeats + clear worker status in the UI, and budget the support
cost.
168     - Middle tier to consider ? "zero-infra BYO": a managed-but-isolated path where the
169     storage stays in the user's bucket (signed URLs) while compute runs on our
GPUs
170     in a strictly isolated, per-tenant, ephemeral sandbox (nothing persists our side).
171     This gives the BYO privacy benefit (raw video never persists on our infra)
without
172     the user running a worker ? a pragmatic default between full-hosted and full-BYO.
173
174     The pipeline code is unchanged; only where it's invoked and how it gets bytes vary.
175
176     ### 3.3 Reuse, don't reinvent
177     The existing segmenter / provider / validator / sport / annotation registries
178     are the proven precedent and stay as the pipeline layer. The annotation registry +
179     candidate_segments table are also the natural seam for the coach-correction / golden
180     -set loop in the assistant-coach phase ($6, P4).
181
182     ## 4. Multi-tenant data model & serving layer
183

```

```

184     ### 4.1 Entities ? generic core, with the coach model as an optional overlay
185     The foundational schema is deliberately **domain-neutral** so we don't overfit to the
186     coach mindset ? the coach/athlete framing is a **market-fit hypothesis that will
iterate**
187     (see C13 / PMF), not a load-bearing assumption. **Generic core:**
188     ```
189     User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
190             ???< PlaySegment ??< Event
191     Video / Highlight ??< Job (processing record: status, compute backend, timings)
192     ```
193     - **Generic & re-targetable:** a `VideoCollection` groups a user's videos (a season, an
194       event, a "folder") with no assumption of a coach or academy ? a power user, a coach,
or a
195       parent are all just `User`s with collections.
196     - **Highlights are first-class** (today compile only writes a file): a `highlights`
table
197       (segment ids + output `StorageRef` + params + created_at) powers "see past videos and
198       highlights by history," and **`HighlightMetadata`** carries the **rich semantic tags**
199       (shot type, rally outcome, etc. ? see $6 metadata stage) plus titles / ratings /
notes.
200     - **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
201     - **Coach overlay (optional, additive):** for the academy/coach GTM, layer
202       `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an
`athlete_id`
203       tag on collections/videos) **without** baking it into the core tables ? so the schema
204       stays usable for solo power users and the coach model can evolve, or be dropped, as
PMF
205       dictates.
206     - **History/serving API:** list collections/videos/highlights per user (and, with the
207       overlay, by athlete/coach/date); **query highlights by `HighlightMetadata`** (e.g.
shot
208       type / outcome); re-stitch from stored segments without reprocessing.
209
210     ### 4.2 Tenancy isolation ? options & recommendation
211     | Model | Isolation | Ops cost | Best for |
212     |---|---|---|---|
213     | **Shared DB + `tenant_id`** (row-level, ideally Postgres **RLS**) | logical | low |
**Hosted SaaS, many small tenants** ? **recommended default** |
214     | Schema-per-tenant | medium | medium | tens?hundreds of mid tenants |
215     | **DB-per-tenant** (incl. **SQLite-per-tenant**) | strong/physical | higher at scale |
**self-hosted single-academy**, or high-isolation enterprise |
216
217     **Recommendation:** for the hosted product, **migrate SQLite ? Postgres with shared-DB +
218     **`tenant_id` + Row-Level Security**; keep **SQLite-per-tenant** as the **self-hosted /
219     **single-academy** deployment (it reuses today's code almost verbatim). Either way,
220     **abstract DB access behind a repository layer now** so the engine is swappable, and
221     graduate the `PRAGMA user_version` migration discipline to **Alembic** under Postgres.
222
223     ## 5. Orchestration / async jobs
224
225     This is the serving layer that makes multi-user real. It is the **multi-user superset of
226     [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) #16** (async job model) ?
build
227     that first, single-tenant, then generalize.
228
229     - **Job record** (`jobs` table): `id, tenant_id, video_id, type (index|compile|analyze),
230       status (queued|running|done|failed), progress, compute_backend, result_ref, error,
231       timestamps`. `/api/process` returns **202 + job_id**; `/api/jobs/{id}` polls.
232     - **Idempotency** keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the
233       content-hash discipline already in `compute_video_id`.
234     - **Dispatch:** the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs
235       for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,
236       writes artifacts + DB rows. **Observability reports** (already per-video) become the
237       job artifact surfaced in history.
238     - **Queue/worker backend ? a decision to prototype in #16, *not* a default here.** The

```

239 *requirements* are firm ? ****idempotency****, ****resumability**** (a 40-min GPU job mustn't
240 restart on a worker blip), ****progress states**** ? which rule out fire-and-forget
Celery/RQ

241 for the long jobs. The ***choice*** should be made by prototyping in the
242 [``DEFERRED_HARDENING_PLAN.md``](DEFERRED_HARDENING_PLAN.md) #16 slice, weighing:
243 - ****Start simplest:**** ****Arq**** (async-native, natural FastAPI fit) or even a ****plain**
244 Postgres-backed queue + worker****** ? likely sufficient for the ***first*** single-tenant
245 async slice, with the lowest operational complexity and the easiest debugging.
246 - ****Durable-execution upgrade if/when needed:**** ****Hatchet**** (Postgres-backed, per-step
247 retries/timeouts/concurrency; pairs neatly with the pull-based worker) ? attractive
248 because it adds ****no new datastore**** once we're on Postgres, but it ***is*** a new
249 dependency + operational/debugging surface; adopt only when long-job resumability/
250 observability demands it.
251 - ****Heaviest:**** ****Temporal**** ? most mature durable workflows, operationally heavy;
252 reserve for genuinely complex multi-stage pipelines at scale.

253
254 Record the decision criteria in #16: operational complexity, maturity, ****debugging**
255 experience for long GPU jobs******, and whether durability is worth a new dependency.
256 ****Bias:** start with the simplest thing that gives retries + status; graduate only on
257 evidence.******

258
259 ## 5A. Owned generative model (the video-QA assistant) & the data flywheel
260

261 The north-star capability: an ****owned, self-hostable generative video model**** ?
262 fine-tuned on sports video (including ****consented, user-contributed**** footage) ? that
263 ****answers questions about rallies, shots, and technique****. This is the conversational
264 brain of the "assistant coach." Crucially, it does ****not**** replace the known stack; it
265 is introduced ****behind the same abstraction, with the paid API as an always-ready**
266 **fallback****.

267
268 > ****The moat is *not* the model.**** Anyone can download Qwen-VL/Gemma. The defensible
269 > assets are **** (a) the consented, badminton-specific video?QA dataset**** and **** (b) the**
270 > **eval harness****. Build and message ***those*** ? not "we own an LLM."

271
272 This conversational surface (NL queries + coach-mode recommendations) must be
****localized**

273 from the start****** for the Bangalore beachhead ? the query ***and its answer*** in the user's
274 language. The locale foundation and the templated-answer approach are specified in
275 [``I18N_PLAN.md``](I18N_PLAN.md) (phase P4 ties to this section).
276

277 > ****? Committed guardrail (legal, non-negotiable).**** We respect the licensing findings
in
278 > full. ****Paid frontier models (Gemini / OpenAI / Anthropic) are used for *inference /**
279 > **serving / fallback* only ? never as a *training* data source.**** Harvesting their
outputs
280 > to train our owned model would breach their "no competing model" terms (a
terms/copyright
281 > violation), so it is prohibited in both engineering and legal process. ****Training**
teachers
282 > are open, Apache-2.0 models only****** (Qwen-VL / InternVL / Gemma 4). Detail in the
283 > distillation constraint below.
284

285 ### Core principle: AI is a pluggable provider with a paid fallback
286 The codebase already abstracts the cloud AI behind ``provider_registry`` /
287 ``AIVideoProvider`` (``backend/providers/``). Make that a ****first-class platform**
principle******:

288 - Every generative capability routes through the ****provider registry****.
289 - The ****owned model**** plugs in as a new provider (``owned_vlm``); ****Gemini (paid, via**
290 Vertex AI)****** stays registered as the ****fallback**** (failover on error/low-confidence,
291 or as the quality ceiling/teacher reference).
292 - This lets us ****ship today on the known paid stack**** and ****swap in the owned model when**
293 its quality clears a measured bar****** ? with zero architectural churn and the paid path
294 always one config flip away. (Mirrors the COMMERCIALIZATION C4 A/B/C strategy: the
295 owned model is the long-run "(A) eliminate the per-call dependency"; paid is "(C)".)
296

```

297     ### The model / training layer (new sub-layer, fed by the scaffolding)
298     ...
299     Data curation      Training      Eval      Serving
300     (consented user vid (fine-tune an open, (extend the existing (self-host inference
301     + golden set + commercially-usable eval harness to QA as a provider /
302     rally/segment/event base VLM via quality + the ComputeBackend;
303     annotations ? LoRA/QLoRA) regression gate) paid fallback)
304     video-QA pairs)
305     ...
306     This layer **reuses every other layer**: **Storage** holds the training corpus;
307     **Data** supplies labels (the `annotation` registry + `candidate_segments` +
308     golden set are already the labeling seam); **Compute/Orchestration** runs training and
309     inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
310     **eval harness** already in `backend/eval/` extends to grade QA quality. **In other
311     words, the rock-solid scaffolding is exactly what makes the model flywheel possible ** ?
312     which is why scaffolding comes first (see §6).
313
314     ### Base model & training (Apache-2.0 only ? licensing is decisive)
315     **Recommended near-term base: `Qwen2.5-VL-7B-Instruct`** ? Apache-2.0, native long-video
316     (dynamic-FPS sampling, temporal mRoPE), mature LoRA tooling. Upgrade path
317     **Qwen3-VL-4B/8B (Apache-2.0)**; **Gemma 4 multimodal (Apache-2.0, 2026)** is a clean
318     Google-stack alternative to evaluate.
319     - ?? **Lock to Apache-2.0-clean bases** so you can fine-tune, **keep weights private,
320     gate behind subscriptions, and never owe open-sourcing or branding**. (Note Qwen2.5-VL
321     is Apache **only at 7B/32B** ? 3B/72B use the custom Qwen license.)
322     - ? **Reject for the flagship:** **Gemma 3** ? custom terms with a **viral derivative
323     clause** (anything fine-tuned on it, *or even trained on its synthetic outputs*, stays
324     permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch
325     to
326     Apache-2.0 removes this**. **Llama 3.2 Vision** ? custom license forces a "Llama-?"
327     model name + "Built with Llama" badge and a 700M-MAU ceiling. **`gpt-oss`** ?
Apache-2.0
327     but **text-only**, so usable only as an optional text-reasoning layer, never the video
328     encoder.
329     - **Training method:** **QLoRA** (? r=64, ?=128) on the 7B ? fits one prosumer GPU and
330     avoids catastrophic forgetting of general video skills. Build the dataset by
converting
331     the existing rally/shot/segment annotations + golden set into templated **clip?QA
332     pairs**, expand synthetically with **open-model teachers only** (see constraint
below),
333     and hold out a **golden QA eval** that gates every fine-tune. A few hundred gold pairs
?
334     **5k?20k** curated is a realistic band for this narrow domain.
335     - **Serving economics:** a 7B VLM ? 14 GB VRAM (FP16) / ~3.5 GB (INT4); a ?16 GB GPU is
a
336     comfortable serve target (vLLM for throughput). Self-hosting wins **only** once you
have
337     sustained video-QA volume **+** a defensible consented dataset **+** real per-call API
338     cost pain ? below that, the paid API is cheaper than a standing GPU and the second
stack
339     is overhead. Hence: **paid-primary now ? owned-primary / paid-fallback later.**
340
341     Base-model licensing + training-data policy are tracked as **COMMERCIALIZATION C14**.
342
343     ### Training backend ? managed service vs self-managed (the fine-tune "forge")
344     *Where* you run the LoRA/RL fine-tune. Weigh on **video support, weight
export/ownership,
345     smallest servable model, RL support, no-train DPA, and infra burden**.
346
347     **Decisive finding (researched 2026-06):** managed fine-tuning services are
348     **image-based** (JSONL + base64 images) ? for video you pre-sample frames and feed them
349     as image sequences. **Only the self-managed `ms-swift` explicitly advertises *native*
350     video (+audio) training** So for true video-QA, a self-managed framework is the
351     strongest technical fit ? and it also lets you fine-tune the **small 7B** (cheap to
352     serve) and keep full ownership. Managed services trade some of that for zero infra.

```

```

353
354 | Option | Type | Video / VLM support | Smallest VL | RL | Export weights | Best for |
355 |---|---|---|---|---|---|---|
356 | **ms-swift** (ModelScope) | Self-managed (rented/own GPU) | **Native video+audio**;
Qwen3-VL/Omni, 300+ MLLMs | any incl **7B** | ? DPO/GRPO | ? full ownership | **? video-QA;
slots into BYO-compute ($3.2)** |
357 | **LLaMA-Factory** | Self-managed | Qwen3-VL + 100+ VLMs; video datasets | any incl 7B
| ? | ? | popular, well-documented |
358 | **Unsloth** | Self-managed (1-GPU optimized) | Qwen3-VL LoRA (vision+lang; frames) |
any incl 7B | ? GRPO | ? | cheapest first runs |
359 | **Tinker** (Thinking Machines) | Managed API (low-level) | Qwen3-VL **30B-A3B+ only**;
image input (video **unverified**) | 30B-A3B MoE | ? PPO/GRPO, custom loss | ? adapters + ckpt;
no-train ToS | zero-infra RL/research at MoE scale |
360 | **Together AI** | Managed (tune + serve) | Qwen3-VL / Gemma-3 / Llama-4; image data |
verify | partial | ? adapters downloadable | managed Qwen3-VL + serving |
361 | **Fireworks AI** | Managed (tune + serve) | Full Qwen2.5-VL (3B?72B SFT; LoRA@32B +
LoRA-upload) | **7B** | partial | ?? verify export (lock-in risk) | managed *small*-VL + serving
|
362 | **Predibase** (Rubrik) | Managed *** VPC self-host** | VLM instruct-tune; **LoRAX**
multi-adapter | verify | ? (VPC) | ? **RFT/GRPO** flagship | RL "creativity" + multi-adapter +
privacy |
363 | ~~OpenAI GPT-4o / Gemini tuning~~ | Closed | ? | ? | ? | ? no export | **non-fit ?
violates "own the model"** |
364
365 *(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full
366 control but are only worth it if you're already on that stack.)*
367
368 **Recommendation:**
369 - **Early iteration / true video-QA ?** self-managed **`ms-swift`** (or LLaMA-Factory /
370 Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud):
native
371 video, any size incl. **7B** (cheap to serve), full ownership, and it drops into the
372 **ComputeBackend / BYO-compute** seam ($3.2). **Likely a better fit than Tinker** for
373 the video requirement *and* serving economics.
374 - **Managed, no GPU ops ?** **Together AI** (Qwen3-VL + downloadable adapters + serving)
375 or **Fireworks** (full Qwen2.5-VL incl. 7B + serving) ? confirm frame/video support
and
376 adapter export.
377 - **If RL "coaching creativity" + per-sport multi-adapter is central ?** **Predibase**
378 (RFT + LoRAX, VPC self-host for the biometric/privacy posture) or **Tinker** (lowest-
379 level RL primitives).
380 - **Tinker** remains a strong zero-infra RL-research option with clean weight export ?
just
381 not the cheapest path to a *small video* model (its VL floor is 30B-A3B).
382 - **Portability is the hedge:** you own the **data + recipe + exported weights**, so the
383 backend is swappable ? same ethos as the provider/registry pattern. Don't get locked
in.
384
385 **Verify before committing (any backend):** (1) **native video / frame-sequence**
386 ingestion + max frames per example; (2) **weight export** + a **commercial DPA**
covering
387 consented, minors-excluded video (biometric, $7); (3) the **inference plan** for the
size
388 you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).
389
390 ### Training harness ? make it agent-runnable (nanochat-style operability)
391 The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled
video
392 and get a trustworthy ship / no-ship verdict, unattended.** Karpathy's **nanochat** is
the
393 reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
394 tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
395 score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
396 five properties are exactly what make training agent-runnable, and we copy them:
397 **(1)** one command, end-to-end; **(2)** one complexity dial (sane auto-derived

```



```

defaults);
398  ** (3) ** an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
399  ** (4) ** reproducible + deterministic + cheap; ** (5) ** minimal/readable.
400
401  **Engine (don't reinvent):** ms-swift already delivers this for VLMs ? CLI-driven,
402  **native video**, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
403  `args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **nanoVLM** ?
404  ~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)
405
406  **The harness = thin glue over components we already have** ? one declarative,
407  agent-invokable job (call it `train_speedrun`):
408  1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry
+
409  `candidate_segments` + golden set ? the labeling seam in `backend/`).
410  2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an
orchestration
411  job (§5 / DEFERRED #16): one config, idempotent, logged.
412  3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness
to
413  **QA quality**; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
414  4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays)
**only
415  if it beats the bar**.
416
417  So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
418  provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
419  wrapper; ms-swift is the engine; the **golden QA eval is the report card**.
420
421  > **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA
eval
422  > set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic
and
423  > trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
424  > optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)
425
426  ### ?? Teacher / distillation constraint (the single biggest legal landmine)
427  You **cannot** train the owned model on outputs from the paid frontier APIs ? a general
428  video-QA assistant is squarely a "competing model": **Gemini** ("may not use the
429  Services to develop models that compete"), **OpenAI** ("may not use Output to develop
430  models that compete"), **Anthropic** (bars using Outputs to train competing models). Two
431  safe lanes: ** (1) ** generate synthetic QA with **open-model teachers only** (Qwen-VL /
432  InternVL / Gemma 4 ? Apache-clean); ** (2) ** keep the paid API strictly as a **runtime
433  fallback** ? serving answers is fine, *harvesting its outputs to train the model that
434  replaces it is not*. Write a hard ***"no training on competitor-API outputs"*** rule into
435  eng + legal process. (This makes the ROADMAP's "Gemini as offline teacher only" precise:
436  the teacher is *open models*, not the paid APIs.)
437
438  ### Data flywheel, personalization & consent tiers
439  Two complementary loops, each with its **own** opt-in (training consent is separate from
440  processing consent ? §7):
441
442  - **Global model (shared ? raises the floor).** Train the base owned model on an
443  **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's
bucket
444  (preserves "your video, your storage"). Flywheel: *more consented, labeled matches ? a
445  better shared model ? better highlights/answers ? more reason to contribute.* Higher
446  consent bar (pooled data, biometric/minor care ? §7); **minors excluded by default**.
447
448  - **Per-user adapters (personalized ? raises the ceiling) ? owner hunch, under
exploration.**
449  A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own
450  consented videos*** on top of the shared base, served via **multi-LoRA** (LoRAX /
451  Together / Predibase). Why it's compelling:
452  - **Precision:** adapts to *that user's* court, camera angle, lighting, and players'
453  styles ? materially better detection/highlights for them.

```

```

454 - **Cleanest consent story:** "improve **your** detection using **your** data" is
far
455 lower-friction and lower-risk than contributing to a global model ? the data
improves
456 **only the user's own adapter**, with **no cross-user mixing**, sidestepping much of
the
457 pooled-corpus biometric/minor entanglement. Reinforces the differentiator: *your
video,
458 your storage, your compute ? and now **your model**.* With BYO-compute the adapter
can be
459 trained/served on the user's own GPU and **never leave their boundary**; **deletion
=
460 drop the adapter** (clean erasure).
461 - **Open questions (still a hunch):** cold-start (need enough of the user's labeled
data
462 before a personal adapter beats the base ? fall back to the shared model meanwhile);
a
463 **per-user held-out eval** to prove the adapter helps (don't ship a regression);
base +
464 optional global-adapter + per-user-adapter stacking; per-user training cost at
scale.
465 Worth a spike once the harness + golden eval exist.
466
467 **Two consent tiers, then:** (a) "improve **my own** detection with **my** data"
468 (per-user, low friction) and (b) "contribute to the **shared** model" (global,
higher
469 bar). **Default for both is off.**
470
471 ## 6. Phased roadmap
472
473 | Phase | Deliverable | Build / reuse |
474 |---|---|---|
475 | **P0** | ? **DONE ? Async job model, single-tenant** (DEFERRED #16) | `jobs` table via
`user_version` ladder; single-worker `ThreadPoolExecutor`; `/api/process` ? 202 + job_id; `GET
/api/jobs/{id}` status |
476 | **P1** | **Identity + multi-tenant data model + upload + highlight persistence +
history** | auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS;
`highlights` table; repository layer; history API + UI |
477 | **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry;
`local`? `gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
|
478 | **P3** | **`ComputeBackend` registry + BYO-compute** | `local`/`hosted`/`byo_worker`;
pull-based worker agent; control/data-plane split |
479 | **P4** | **Rich rally/shot semantic metadata & tagging** *(the next value stage after
solid rally detection + highlights)* | shot type (smash / backhand drive / net), rally
**outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ?
query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden
set via the `annotation` registry + `candidate_segments` |
480 | **P5** | **Owned generative video-QA model** ($5A) | global flywheel **per-user
adapters** ; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as
`owned_vlm` **provider with Gemini paid fallback** ; extend the eval harness to QA quality |
481 | **P6** *(visionary)* | **Gait / biomechanics under a supervising coach** |
deliberately **back-burner** ; technique feedback; highest Art. 9 bar ? gate behind explicit
biometric + parental consent ($7) |
482
483 P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3
484 deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the
485 immediate next value stage once rally detection + highlights are solid** ? e.g.
**"rallies
486 that ended in a smash or backhand drive," "rallies where the court was open but the
487 player made an unforced error." **P5** owns the model; **P6 (gait) is the visionary
488 horizon, intentionally deferred.**
489 **Sequencing principle (owner directive):** make the scaffolding (P0?P4) **rock-solid
490 first**, keep the **known paid stack as an always-ready fallback**, and only then shift
491 focus to **owned-model quality** (P5). The provider registry keeps the paid path one

```

```

492     config flip away.
493
494     ## 7. Privacy & legal (gait raises the bar ? design for it now)
495
496     - **The load-bearing nuance:** biometric data is special-category **only when used to
497       *uniquely identify* a person** (GDPR Art. 9). **Highlight/shot detection on players a
498       coach already knows is general analytics ? generally *not* Art. 9.** But
499       **gait/biomechanics that builds a per-athlete identifying template tips into Art. 9**
500       and needs **explicit, granular consent** referencing biometric data. Engineer gait
501       with that boundary (consent gate, purpose limitation, no cross-account
identification);
502       decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB
503       Guidelines 3/2019 on video devices apply.)
504     - **Minors (e.g. academies).** Parental/guardian consent and safeguarding are first-
order,
505     not an afterthought. Model consent explicitly per relationship (user?subject,
coach?athlete
506     where the overlay applies, parent?minor).
507     - **BYO-storage is the central privacy/liability lever:** if raw video stays in the
508     tenant's own bucket/Drive and we process via signed URL or their compute, we minimize
509     what we hold ? shrinking breach surface, residency, and erasure obligations.
510     - **Paid-LLM no-train (C4):** use **Gemini via Vertex AI** (contractual no-training),
511     **not the consumer/free tier**; set short/zero retention, document it in the DPA, and
512     surface "your footage is never used to train AI" as a trust feature for academies. For
513     the gait phase, prefer keeping biometric inference off third-party APIs entirely.
514     - **Training consent is a distinct, higher bar (for the owned model, §5A).** Training on
515     user video needs a **separate, explicit, opt-in training licence** ? not bundled with
516     processing consent ? and it **compounds** the biometric (Art. 9) and minor-consent
517     issues above. Train **only on explicitly contributed/licensed video**; never train on
518     what merely passes through a tenant's bucket. **Exclude minors' video from the
training
519     pool by default**, keep the training corpus a **separate, firewalled, opt-in sub-
pool**,
520     and log **per-clip consent/provenance** for revocation + audit. Incentives must not
make
521     consent non-free (the product stays fully usable without contributing). And the
522     **teacher-distillation rule** (§5A): never train on paid-LLM outputs. **Two consent
tiers
523     (§5A):** *(a)* "improve **my own** detection with **my** data" ? a per-user adapter
trained
524     only on the user's own video (lowest friction; no cross-user mixing; deletion = drop
the
525     adapter); *(b)* "contribute to the **shared** model" ? pooled corpus, higher bar.
Default
526     for both is **off**.
527     - **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across
DB +
528     storage artifacts (the `ON DELETE CASCADE` discipline already in `database.py` extends
here).
529
530     ## 8. Key decisions for the owner (the forks)
531
532     1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1)
with
533     **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-
user/enterprise
534     option ? it's the highest-complexity, highest-security item.
535     2. **Database now or later.** Move to **Postgres at P1** for the hosted path, or stay
536     SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access
537     regardless.)
538     3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).
539     4. **Orchestration tech** (see research recommendation) ? bias to the simplest that
gives
540     retries + status.
541     5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-

```

max).

6. ****How far to push BYO-compute**** given its security/ops complexity.
7. ****Owned-model base + licensing (\$5A).**** Which commercial-OK video VLM to build on (Qwen-VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial derivative is permitted ? see COMMERCIALIZATION C14.
8. ****Training-data sourcing & consent.**** Opt-in contribution terms, incentives, and the teacher-distillation constraint (whether paid-LLM outputs may seed the corpus).
9. ****When to start the owned model**** ? strictly after the scaffolding + flywheel are solid (the explicit directive), with the paid provider as the standing fallback.

****Risks:**** scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS + tests); BYO-compute security; ****biometric/minor liability**** (get this right before any gait feature ships); ****training on user video without proper opt-in/licence****, and ****base-model / teacher-distillation licence violations****; cost of hosted GPU (training *and* inference); ****premature model-building**** before data/eval are ready (mitigated by keeping the paid fallback and gating P5 behind the flywheel).

9. Sources

External best-practices research (2026-06-07) backing the recommendations above. Each underlying claim in the research was tagged fact-vs-synthesis; figures are as-of and should be re-validated before committing budget.

****Storage abstraction:**** [fsspec](https://fsspec.com/) · [cloudpathlib](https://github.com/drivendataorg/cloudpathlib) · [smart_open](https://github.com/piskvorky/smart_open) · [Apache Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) · [signed URLs across clouds](https://medium.com/@shaikhsarwan49/signed-urls-in-aws-gcp-and-azure-a-beginner-friendly-guide-e2e549425865)

****Multi-tenancy:**** [Bytebase: multi-tenant DB patterns](https://www.bytebase.com/blog/multi-tenant-database-architecture-patterns-explained/) · [Hunchbite: RLS vs schema-per-tenant](https://hunchbite.com/guides/multi-tenant-saas-architecture) · [PlanetScale: tenancy in Postgres](https://planetscale.com/blog/approaches-to-tenancy-in-postgres) · [DEV: DB-per-tenant vs shared schema](https://dev.to/young_gao/multi-tenant-architecture-database-per-tenant-vs-shared-schema-1n2e)

****BYO-compute / orchestration:**** [Pinecone BYOC](https://www.pinecone.io/blog/byoc/) · [Nuon: control/data-plane](https://nuon.co/blog/byoc-control-plane-data-plane-architectures) · [GitHub self-hosted runners (ARC)](https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners-with-actions-runner-controller/about-actions-runner-controller) · [SkyPilot](https://github.com/skypilot-org/skypilot) · [SkyPilot AI control plane](https://blog.skypilot.co/ai-job-orchestration-pt2-ai-control-plane/) · [Modal (AWS Marketplace)](https://aws.amazon.com/marketplace/pp/prodview-j727623xqhh2k) · [Northflank: Anyscale alternatives](https://northflank.com/blog/anyscale-alternatives-for-ai-ml-model-deployment) · [DevPro: Python task queues 2025](https://devproportal.com/languages/python/python-background-tasks-celery-rq-dramatiq-comparison-2025/) · [Judoscale: choosing a task queue](https://judoscale.com/blog/choose-python-task-queue) · [Hatchet vs Temporal](https://hatchet.run/versus/hatchet-vs-temporal) · [Hatchet](https://github.com/hatchet-dev/hatchet)

****Privacy / legal:**** [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [GDPR Local: biometric compliance](https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/) · [Legiscope: Art. 9](https://www.legiscope.com/blog/gdpr-article-9-special-categories.html) · [EDPB Guidelines 3/2019 (video)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_201903_video_devices_en_0.pdf) · [Fox Rothschild: EDPB video guidance](https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance) · [ICO: children & UK GDPR](https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/children-and-the-uk-gdpr-old/what-are-the-rules-about-an-iss-and-consent/) · [EDPB Guidelines 05/2020 (consent)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_202005_consent_en.pdf) · [AI data-retention compared](https://ax-sentinel.com/blog/ai-data-retention-policies-compared) · [Xenoss:

enterprise LLM platforms](https://xenoss.io/blog/openai-vs-anthropic-vs-google-gemini-enterprise-llm-platform-guide)

571

572 **Comparable products:** [Veo](https://www.veo.com/) · [Veo Help Center](https://support.veo.com/hc/en-us/articles/4459159526929-Overview-of-cam-veo-co-Record-manage-and-upload-footage-with-your-Veo-Cam-1) · [Latterly: Hudl competitors](https://www.latterly.org/hudl-competitors/) · [Apple: Behind the Design ? SwingVision](https://developer.apple.com/news/?id=0pg4dthn) · [SwingVision FAQ](https://swing.vision/faq) · [Sportsvisio: coach video-analysis comparison](https://www.sportsvisio.com/stories/video-analysis-software-for-coaches-complete-comparison)

573

574 **Training backends & harness (\$5A):** [karpathy/nanochat (harness reference)](https://github.com/karpathy/nanochat) · [HF nanoVLM (minimal VLM)](https://github.com/huggingface/nanoVLM) · [nanoVLM blog](https://huggingface.co/blog/nanovlm) · [Tinker (GA + vision)](https://thinkingmachines.ai/blog/tinker-general-availability/) · [Tinker model lineup](https://tinker-docs.thinkingmachines.ai/model-lineup) · [Tinker ToS (weights/no-train)](https://thinkingmachines.ai/legal/tos/) · [Fireworks VLM fine-tuning](https://fireworks.ai/blog/vlm-tuning) · [Together AI fine-tuning](https://www.together.ai/fine-tuning) · [Predibase: fine-tune & serve VLMs](https://predibase.com/blog/how-to-fine-tune-and-serve-vlms-in-predibase) · [Predibase RFT/GRPO](https://docs.predibase.com/user-guide/fine-tuning/grpo) · [ms-swift (native video/audio)](https://github.com/modelscope/ms-swift) · [LLaMA-Factory (VLMs)](https://github.com/hiyouga/LlamaFactory) · [Unsloth (Qwen3-VL)](https://qwen.readthedocs.io/en/latest/training/unsloth.html) · [Fine-tuning tools 2026 (LoRA/QLoRA)](https://www.index.dev/blog/top-ai-fine-tuning-tools-lora-vs-qlora-vs-full)

575

576 **Owned model & licensing (\$5A):** [Qwen2.5-VL](https://qwenlm.github.io/blog/qwen2.5-vl/) · [Qwen2.5 license split](https://qwenlm.github.io/blog/qwen2.5/) · [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL) · [Gemma 3 (multimodal)](https://huggingface.co/blog/gemma3) · [Gemma Terms of Use](https://ai.google.dev/gemma/terms) · [Gemma license-risk analysis](https://wcr.legal/google-gemma-license-risks/) · [Gemma 4 / Apache-2.0](https://www.mindstudio.ai/blog/what-is-gemma-4-apache-2-license-commercial-ai-deployment) · [Llama 3.2 license (700M MAU + naming)](https://www.llama.com/llama3_2/license/) · [gpt-oss (text-only, Apache-2.0)](https://openai.com/index/introducing-gpt-oss/) · [Gemini API terms (compete clause)](https://ai.google.dev/gemini-api/terms) · [OpenAI terms](https://openai.com/policies/row-terms-of-use/) · [Anthropic: outputs & training](https://support.claude.com/en/articles/12326764-can-i-use-my-outputs-to-train-an-ai-model) · [QLoRA practical](https://apxml.com/courses/fine-tuning-adapting-large-language-models/chapter-4-parameter-efficient-fine-tuning/practice-qlora-fine-tuning) · [VLM LoRA cookbook](https://huggingface.co/learn/cookbook/en/fine_tuning_vlm_trl) · [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [athletes & biometric data](https://www.architectureandgovernance.com/applications-technology/biometric-data-and-athletes-privacy-law-and-compliance-implications/) · [federated learning + DP](https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/) · [self-hosted LLM VRAM/cost](https://www.gpu-mart.com/guides/self-hosted-llm) · [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing)

577

578 ## See also

579 - COMMERCIALIZATION.md ? C12 (SaaS foundations), C4 (paid-LLM), economics.

580 - DEFERRED_HARDENING_PLAN.md ? #16 async job model (the P0 infra slice), #7 packaging.

581 - PMF_MARKET_RESEARCH.md ? academy/coach beachhead (who this serves).

582 - CODE_MAP.md ? current architecture + the registry extension points.

583 - ROADMAP.md ? offline self-improving rally detection (the pipeline's own trajectory).

584

5. assistant (2026-06-19T19:17:40.722Z)

This doc anticipates the idea well at the storage/registry level. Let me verify two specific

claims: whether the "watch a folder / autosync" pull-trigger pattern is anticipated (the doc's ingestion language), and what DEFERRED_HARDENING_PLAN #16 actually says about the job model. Let me also check if a sibling overview doc mentions this.

6. user (2026-06-19T19:17:43.281Z)

```
17:principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
48:> first infra slice (an async job model) is the on-hold #16 in
60:| Execution | **async job model shipped** ? `/api/process` returns **202 + job_id**, a
single-worker `ThreadPoolExecutor` drains the `jobs` table, `GET /api/jobs/{id}` polls status |
single-tenant + local-only; can't dispatch to a user's VM/GPU yet |
62:| API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs auth,
per-tenant scoping, async job + history endpoints |
66:`backend/main.py` (endpoints + the async job worker ? `/api/process` ? 202 + job_id,
154: outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most
223:## 5. Orchestration / async jobs
226:[`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16** (async job model) ? build
231: timestamps`. `/api/process` returns **202 + job_id** ; `/api/jobs/{id}` polls.
243: - **Start simplest:** **Arq** (async-native, natural FastAPI fit) or even a **plain
245: async slice, with the lowest operational complexity and the easiest debugging.
386:ingestion + max frames per example; (2) **weight export** + a **commercial DPA** covering
475:| **P0** | ? **DONE ? Async job model, single-tenant** (DEFERRED #16) | `jobs` table via
`user_version` ladder; single-worker `ThreadPoolExecutor`; `/api/process` ? 202 + job_id; `GET
/api/jobs/{id}` status |
580:- [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) ? #16 async job model (the P0
infra slice), #7 packaging.
```

7. user (2026-06-19T19:17:44.723Z)

```
1:# Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging (COMPLETED)
5:- **#16 Async job model ? DONE** (PR #87; hardened by the self-scheduling drain loop in #149).
6:- **#7 Packaging modernization ? DONE** (PR #167: PEP 621 `pyproject.toml`, `setup.py` ?
`scripts/doctor.py`, `indexer` console entry point, CI switched to `pip install -e .[dev]`).
```

8. assistant (2026-06-19T19:17:48.342Z)

Now I have what I need. Let me read the #16 section of DEFERRED_HARDENING_PLAN to confirm the job model's shape and whether any trigger/watch concept appears.

9. user (2026-06-19T19:17:50.611Z)

```
1:# Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging (COMPLETED)
2-
3-***STATUS: COMPLETED** (2026-06-14). This planning doc's three items are all dispatched:
4-
5:- **#16 Async job model ? DONE** (PR #87; hardened by the self-scheduling drain loop in #149).
6:- **#7 Packaging modernization ? DONE** (PR #167: PEP 621 `pyproject.toml`, `setup.py` ?
`scripts/doctor.py`, `indexer` console entry point, CI switched to `pip install -e .[dev]` ).
7-- **#13 Train/eval separation guardrails ? moved to [`BACKLOG.md`](BACKLOG.md)** (?; the
concrete `splits.py` / `assert_disjoint` match-level-split design is preserved there).
8-
```

10. assistant (2026-06-19T19:17:56.947Z)

I have a complete picture. The doc strongly anticipates the storage half of the idea but the "watch ? autosync ? enqueue" trigger half is a genuine gap. Let me record my findings.

11. user (2026-06-19T19:18:41.533Z)

Structured output provided successfully

12. assistant (2026-06-19T19:18:55.315Z)

I've completed the assessment and returned the structured object. Summary of what I found in

`C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM\_ARCHITECTURE.md`:

****The idea splits into two halves with different fits:****

1. *****"Bring your own storage" ? already designed (partial ? yes).***** §3.1's `StorageBackend` registry explicitly enumerates `local`, `gdrive`, `s3`, `gcs`, `azure` (plus Dropbox via `dropboxdrivefs`), built over `fsspec`, with BYO-storage as a first-class property: *****"the tenant registers their own bucket/Drive; raw video never leaves their account."***** The ABC even exposes `list(prefix)` and `stat()` ? the exact primitives a watcher would call. This is roadmap ****P2****.

2. *****"Autosync / watch the folder ? enqueue" ? a genuine gap.***** The doc names *****"ingestion"***** as a decoupled layer (line 17) but ? unlike storage/compute/orchestration ? never gives it a section, interface, or roadmap phase. Grep for `watch|sync|poll|webhook|trigger` returns nothing. The shipped async job model (`#16`, now DONE) is *****pull-triggered by a user POST to `/api/process`*****; nothing auto-creates a job when a file lands. So the watcher would be a new control-plane Sync component that diffs each tenant's `StorageBackend` prefix (or subscribes to GCS Pub/Sub / S3 events / Drive-changes / Graph-delta) and feeds the existing §5 `jobs` table as a new automated ***producer***, reusing the `(tenant\_id, video\_id, pipeline\_version)` idempotency key (§5) for free dedup.

****Key risks the doc surfaces or implies:**** egress/compute-placement tension (hosted GPU downloading from the user's bucket weakens the "never leaves your account" promise ? §3.2's "zero-infra BYO" signed-URL sandbox is the cleanest fit); the OAuth conflict between §3.1's "never persist long-lived keys / narrow `drive.file` scope" and a continuously-watching server needing long-lived/refreshable creds; partial-upload readiness; and not re-ingesting KhelSutra's own reels written back to the same bucket.

Relevant file: `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM\_ARCHITECTURE.md` (§3.1, §5, §6 P2), cross-checked against `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DEFERRED\_HARDENING\_PLAN.md` (#16, now marked COMPLETED).